

PROGRAMAÇÃO FUNCIONAL (T300)	
Nome do Projeto: Calculadora de Calorias	
TERMO DE ABERTURA DO PROJETO	
Professor: Gilson Pereira do Carmo Filho	Semestre: 2026.1
Avaliação: AV3	Pontos: 2

1. Justificativa

À medida que os consumidores se tornam mais preocupados com a saúde, eles querem saber mais sobre o conteúdo dos alimentos que ingerem. Cada alimento é composto por blocos de construção, como carboidratos, minerais e vitaminas. As informações nutricionais são o detalhamento desse conteúdo.

Em um nível básico, essas informações incluem dados sobre macronutrientes e a contagem de calorias dos alimentos. Assim, ao monitorar o consumo calórico, as pessoas podem identificar seus hábitos alimentares, aprender sobre o valor nutricional dos alimentos e fazer escolhas mais conscientes.

Por outro lado, complementarmente, ao monitorar a perda calórica com atividades físicas, você pode identificar quais destas são mais eficazes para você e ajustar a sua rotina para obter melhores resultados. Dessa forma, você se torna mais consciente do seu gasto energético e pode tomar decisões mais assertivas para alcançar seus objetivos de saúde.

Nesse contexto, um aplicativo que registra ganho e perda calóricos permitiria planejar um esquema adequado de alimentação e de exercícios físicos para alcançar os objetivos de saúde, como ganho de massa muscular ou perda de peso, sendo uma ferramenta valiosa para quem busca uma vida mais saudável e um estilo de vida mais ativo.

2. Descrição do Produto

O produto a ser desenvolvido consiste em uma **calculadora de calorias**. Essa calculadora deverá ser uma aplicação composta por um **back-end**, responsável por registrar as transações e manter a base de dados, e por um **front-end**, responsável pela interface com o usuário (entrada/saída de dados).

De modo geral, a calculadora deverá funcionar da seguinte forma: inicialmente, o usuário precisa fornecer, no **front-end** da aplicação, alguns dados pessoais, como altura, peso,

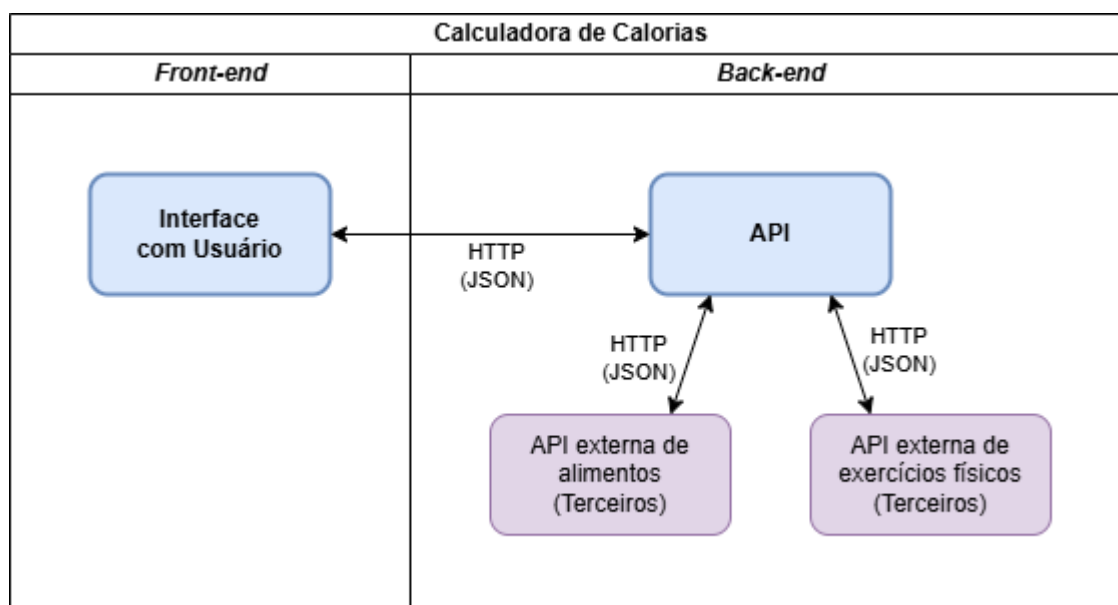
idade e sexo. Esses dados deverão então ser enviados para uma API (*Application Programming Interface*) no *back-end*, a fim de serem cadastrados na base de dados e utilizados quando necessário.

Uma vez que os dados pessoais se encontram cadastrados, o usuário poderá então registrar as transações, ou seja, o consumo de alimentos ou a realização de atividades físicas. Para cada alimento consumido, o usuário também precisará informar uma data e a quantidade consumida. Para cada atividade realizada, o usuário deverá informar uma data e o tempo de duração da atividade.

Os dados acima serão fornecidos pelo usuário no *front-end* da aplicação, que então os enviará para a API no *back-end*, a fim de serem registrados. As calorias associadas ao consumo de um alimento ou à realização de uma atividade física deverão ser obtidas pela própria API, por meio de APIs externas (de terceiros). Essas APIs externas podem ser encontradas em sites de buscas como [Rapid API](#) e [API Ninjas](#).

Finalmente, após calcular a quantidade de calorias do alimento consumido (ganho calórico) ou da atividade física realizada (perda calórica), a transação deverá ser registrada na base de dados.

A figura abaixo ilustra, de forma simplificada, os componentes da aplicação.



Além de permitir o registro de dados pessoais e transações de ganho/perda calóricos, a aplicação também deverá ser capaz de exibir para o usuário um extrato de todas as transações num determinado período, bem como o saldo de calorias.

Em resumo, estas são as operações que o usuário poderá realizar com a aplicação:

1. Cadastrar/consultar dados pessoais (altura, peso, idade e sexo);
2. Registrar consumo de alimento (ganho de caloria);
3. Registrar realização de atividade física (perda de caloria);
4. Consultar extrato de transações (por período);
5. Consultar saldo de calorias (por período).

3. Principais Entregas

As principais entregas do projeto são:

ENTREGAS	REQUISITOS
<i>Back-end</i>	• Dispor de uma API para processar e manter os dados da aplicação; • Usar o HTTP (<i>Hypertext Transfer Protocol</i>) para a comunicação com o <i>front-end</i>, ou seja, a API deverá receber requisições do <i>front-end</i> e enviar respostas para este por meio daquele protocolo; • Usar o formato JSON (<i>JavaScript Object Notation</i>) para a troca de dados entre a API e o <i>front-end</i>; • As transações da aplicação deverão ser representadas por meio de mapas (<i>hash-map</i>), os quais deverão ser armazenados em listas (<i>list</i>); • A base de dados da aplicação deverá ser mantida em memória fazendo uso de átomos (<i>atom</i>), para o gerenciamento de estado; • A API deverá dispor de endpoints para as seguintes operações: ○ Registrar os dados do usuário; ○ Registrar o consumo de um

	determinado alimento; ○ Registrar a realização de um determinado exercício físico; ○ Obter extrato de transações (ganho/perda de calorias); ○ Obter o saldo de calorias.
Front-end	• Prover uma interface com o usuário da aplicação, realizando operações de entrada e saída de dados; • Funcionar como um cliente da API, usando o HTTP (<i>Hypertext Transfer Protocol</i>) para a comunicação com o <i>back-end</i>, ou seja, o <i>front-end</i> deverá enviar requisições para a API e receber respostas deste por meio daquele protocolo; • Usar o formato JSON (<i>JavaScript Object Notation</i>) para a troca de dados entre o <i>front-end</i> e a API; • Deverá ser desenvolvido para uso em uma das seguintes plataformas: <i>desktop</i>, <i>web</i> ou <i>mobile</i>. No caso de <i>desktop</i>, a interface com o usuário poderá ser gráfica ou por linha de comando. Observação: Ferramentas para teste de APIs, como Postman ou Swagger, não serão consideradas como entregas válidas.

4. Estratégia de Condução

- Dúvidas acerca dos requisitos do projeto deverão ser esclarecidas com o professor;
- O código fonte do projeto deverá ser enviado para o AVA até o prazo estipulado para a entrega. Não serão aceitas entregas após o prazo;
- Os projetos deverão ser apresentados pessoalmente ao professor na data definida. Trabalhos feitos em dupla deverão ser apresentados por ambos os alunos;
- A apresentação consistirá em **execução e teste** do aplicativo, seguido de **arguição do professor** sobre o código fonte. **Trabalhos não apresentados receberão a nota ZERO**;
- O código fonte do projeto será submetido a uma ferramenta de verificação de plágio. Qualquer tentativa de cópia do projeto de outro aluno ou da Internet, ou qualquer outra

tentativa de fraudar o projeto, incluindo cópia de trechos do código fonte, resultará em aplicação de nota **ZERO**.

5. Critério de Avaliação

Na avaliação do projeto serão consideradas, além da apresentação, a execução correta das funcionalidades do programa e a conformidade do código fonte ao conteúdo abordado na disciplina.

Com relação à conformidade do código fonte, será observado o uso dos conceitos fundamentais da programação funcional, como funções de ordem superior, pureza, recursão de cauda etc. Não será aceito o uso de funções que simulam loops, como por exemplo, LOOP, WHILE, FOR, DOSEQ e DOTIMES.

A tabela a seguir mostra a pontuação das entregas do projeto.

ENTREGA	PONTOS
<i>Back-end</i>	1,0
<i>Front-end</i>	1,0
TOTAL	2,0

6. Premissas e Restrições

PREMISSAS	RESTRIÇÕES
Os capítulos 9 a 13 do livro Programação Funcional: Uma Introdução em Clojure deverão ser estudados e usados como referência para o desenvolvimento da API.	- O projeto tem um prazo total de 3 (três) semanas; - A plataforma deverá ser totalmente desenvolvida na linguagem Clojure; - O projeto deverá ser realizado em dupla ou individualmente.